- *ip* is the IP address, in dotted form, of the virtual machine to be created.
- *hostname* and *domain* configures the hostname and domain name of the VM.
- *user, name,* and *pass* set the user name, full name, and password of the default user.
- *libvirt qemu:///system* is added to ensure that we will use *virsh* to manage the VM.

## Virtual machine management

*libvirt* is a generic, virtualisation technology-independent API to securely manage local as well as remote domains or VMs. It allows one to create, start, stop, modify, monitor and migrate VMs.

The *libvirt* API provides support for different types of hypervisors. In order to access a VM, either local or remote, a URI is specified, along with the type of hypervisor (*libvirt's* driver and URI approach). This ensures the correct VM is accessed.

*libvirtd* is a daemon to manage QEMU VM instances and *libvirt* virtual networks. *libvirtd* is configured by */etc/libvirt/libvirtd.conf.*

*virsh* is a command-line utility that you can use to issue *libvirt* API calls. Some of the commonly used *virsh* commands are:

- *list* – to list running virtual machines:
```
virsh -c qemu:///system list
```
- *start* – to start a virtual machine
```
virsh -c qemu:///system start  kvm-euc-demo-server-karmic
```
- *autostart* – to start a virtual machine at boot (makes it an upstart job):
```
virsh -c qemu:///system autostart  kvm-euc-demo-server-karmic
```
- *reboot* – to reboot a virtual machine:
```
virsh -c qemu:///system reboot  kvm-euc-demo-server-karmic
```
- *save* – to save the state of a virtual machine to a file in order to be restored later (once saved, the VM will no longer be running):
```
virsh -c qemu:///system save kvm-euc-demo-server-karmic \
kvm-euc-demo-server-karmic.state
```
- *restore* – You can restore a saved virtual machine using:
```
virsh -c qemu:///system restore kvm-euc-demo-server-karmic.state
```
- *shutdown* – to shutdown a virtual machine:
```
virsh -c qemu:///system shutdown  kvm-euc-demo-server-karmic
```

*libvirt* stores its configuration as XML in */etc/libvirt/qemu*. While it is possible to modify the XML directly, it is better to modify it using virsh or *virt-manager*. It is possible to add CPU, memory, add/change a network card used by a VM, etc.

## Virtual networking

There are essentially two different ways for a VM to access the network. The first is to use NAT forwarding or virtual networks. This is provided by default, and traffic is 'NATed' through a virtual bridge device *virbr0* to the network. In this case the VM does not connect directly to the LAN; rather, it connects through the virtual bridge using NAT forwarding. The commands *virsh net-list –all* and *brctl show* can be used to see the details of *virbr0.*

The second way for a VM to access the network is using bridge networking or a shared physical device. The bridge *br0* is configured in */etc/network/interfaces*, and this allows the VM to connect directly to the LAN.

## Virtual Machine Manager

The Virtual Machine Manager is a graphical utility to manage local and remote virtual machines. It is part of the *virt-manager* package (installed using *sudo apt-get install virt-manager*).

To connect to the local *libvirt* service, use the following command:

```
virt-manager -c qemu:///system
```

To connect to the *libvirt* service running on a remote host, use:

```
virt-manager -c qemu+ssh://virtnode1.mydomain.com/system
```

## UEC and Eucalyptus

The private cloud functionality of UEC is provided by Eucalyptus (short for Elastic Computing Architecture for Linking Your Programs to Useful Systems), which is an open source system that helps to transform a group of Linux servers into a cloud computing platform.

### Eucalyptus and the various controllers

In order to understand Eucalyptus better, we would recommend you read an excellent paper called *The Eucalyptus Open-source Cloud-computing System*, available at *http://open.eucalyptus.com/wiki/Presentations*. This paper explains the various controllers as follows:

- Node Controller (NC) controls the execution, inspection, and termination of VM instances on the host where it runs.
- Cluster Controller (CC) gathers information about and schedules VM execution on specific node controllers, and manages the virtual instance network.
- Storage Controller (Walrus) (SC) is a put/get storage service that implements Amazon's S3 interface, providing a mechanism for storing and accessing virtual machine images and user data.
- Cloud Controller (CLC) is the entry-point into the cloud for users and administrators. It queries node managers for information about resources, makes high-level scheduling decisions, and implements them by making requests to cluster controllers.